

1. Objetivos

- Configurar o Player
- Mover o Player e a Câmera
- Animação de Ataque
- Criar a 2ª cena: “Vilarejo”???

2. Configurando o Player

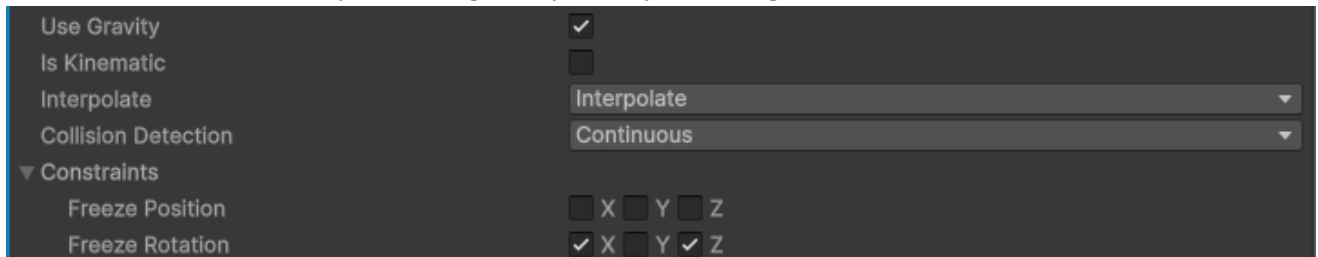
- Importe um guerreiro de Mixamo, por exemplo, **Morak**, sem animações, acertando os materiais se ficar branco.
- Coloque-o num terrain qualquer que tenha diferenças de inclinação no horizonte, para sabermos os giros e os deslocamentos.
- Coloque-o na Hierarchy na posição central, por exemplo:



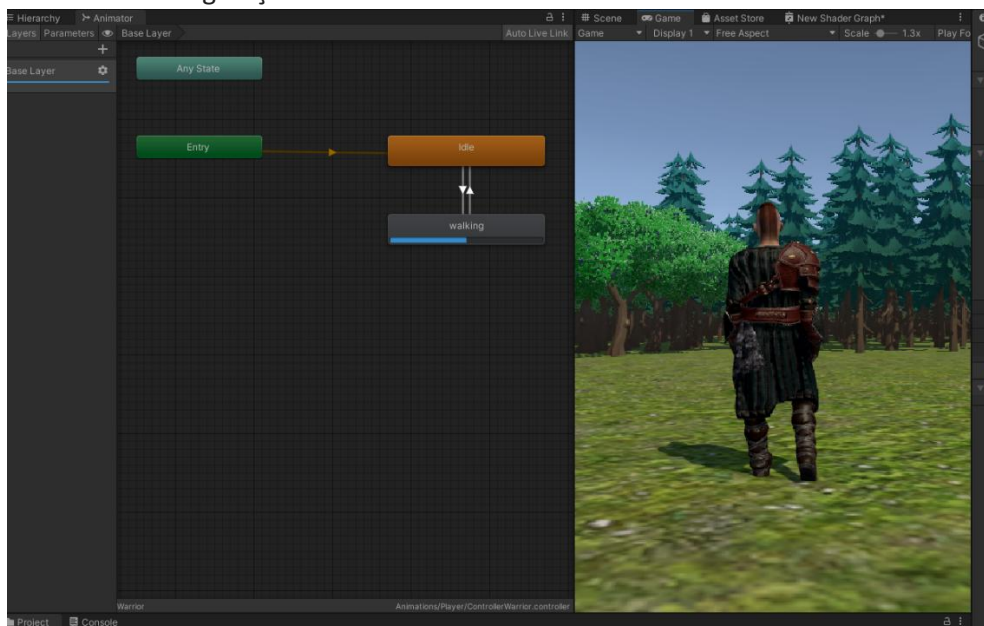
- Adicione um Capsule Collider, nele, ajustando as dimensões, como:



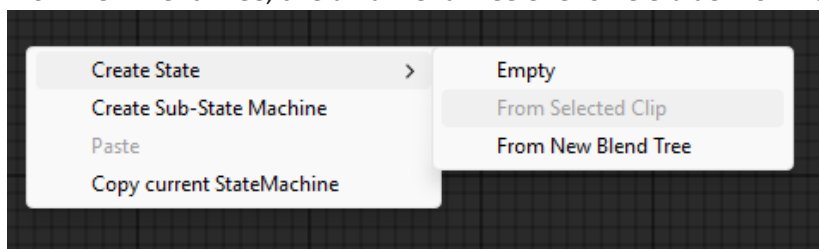
- e. Adicione um componente Rigidbody ao Player e configure:



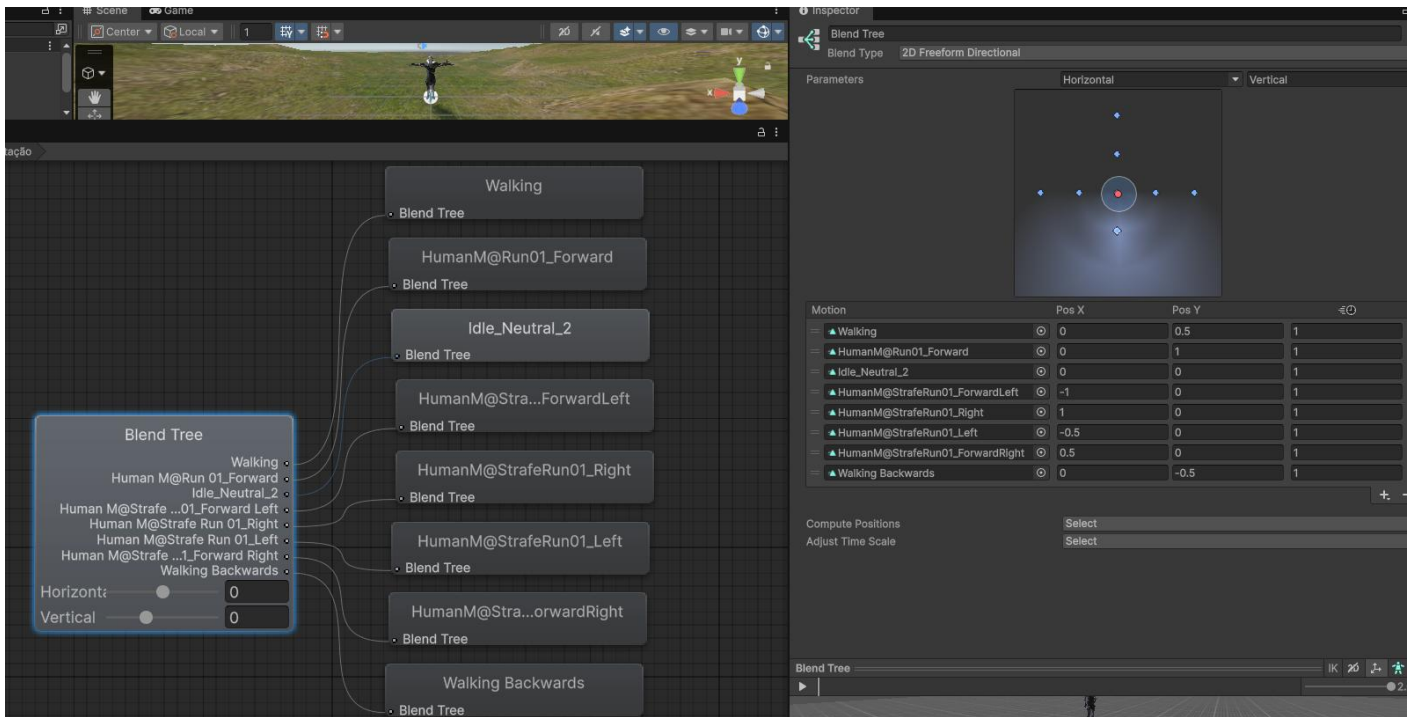
- f. Importe da Asset Store o **Human Melee Animations FREE**.
 g. Crie um Animation Controller para o Player, renomeie de ControllerWarrior.. Arraste o ControllerWarrior para o Player.
 h. Arraste a Main Camera para ser filha de Warrior. Configure-a para uma visão em 3ª Pessoa, com Position XYZ = (0,1,-4).
 i. Importe o Package “AnimationsMale”.
 j. Arraste os estados Idle e walking forward para o ControllerWarrior, e faça as transições para verificar as configurações:



- k. Na área vazia do Animator Controller, clique com o botão direito do mouse e > Create State > From New Blend Tree, crie uma Blend Tree e renomeie-a de Movimentação.



- l. No Animator Controller, em Parametes, clique em + e crie dois float um “Horizontal” e outro “Vertical”.
 m. No pop up do Inspector da Blend Tree escolha o tipo > 2D Freeform Directional, e vá clicando no botão + para adicionar uma a uma a animações “Walking”, “HumanM@Run01_Forward” (do pacote importado da pasta \Kevin Iglesias, etc..(6 desta pasta) e “Walking Backwards”, configurando Pos X e Pos Y para cada animação:



- n. Na Base Layer, coloque esta Blend Tree Movimentação como o estado default.
- o. Mudaremos vários scripts para este Novo Player, e os escreveremos em inglês para diferenciar dos scripts da fase “Despertar”.
- p. Salve esta cena como “Warrior”.
- q. Serão 3 scripts para o Player:
 - i. O primeiro script será “PlayerMovement.cs”:

```

1 using UnityEngine;
  0 references
2 public class PlayerMovement : MonoBehaviour
3 {
4     1 reference
5     public float walkSpeed = 2.5f;
6     1 reference
7     public float runSpeed = 5f;
8     1 reference
9     public float rotationSpeed = 10f;
10    8 references
11    public Transform cameraTransform;
12    5 references
13    public Animator animator;
14    2 references
15    public float animationSmooth = 0.08f;
16    5 references
17    private Rigidbody rb;
18    10 references
19    private Vector2 moveInput;
20    3 references
21    private bool isSprinting;
22    3 references | 3 references
23    private float currentAnimX, currentAnimY;
24    1 reference | 1 reference
25    private float animXVelocity, animYVelocity;
  
```

```

15      0 references
16      private void Awake()
17      {
18          rb = GetComponent<Rigidbody>();
19          if (animator == null)
20          {
21              animator = GetComponentInChildren<Animator>();
22          }
23
24      0 references
25      public void SetMoveInput(Vector2 input)
26      {
27          moveInput = input;
28      }
29
30      0 references
31      public void SetSprint(bool sprinting)
32      {
33          isSprinting = sprinting;
34      }
35
36      0 references
37      private void FixedUpdate()
38      {
39          MovePlayer();
40          RotatePlayer();
41      }
42
43      0 references
44      private void Update()
45      {
46          UpdateAnimator();
47      }

```

```

43      1 reference
44      private void MovePlayer()
45      {
46          Vector3 forward = cameraTransform != null ? cameraTransform.forward : Vector3.forward;
47          Vector3 right = cameraTransform != null ? cameraTransform.right : Vector3.right;
48
49          forward.y = 0f;
50          right.y = 0f;
51          forward.Normalize();
52          right.Normalize();
53
54          Vector3 move = forward * moveInput.y + right * moveInput.x;
55          float currentSpeed = isSprinting ? runSpeed : walkSpeed;
56          Vector3 velocity = new Vector3(
57              move.x * currentSpeed, rb.linearVelocity.y, move.z * currentSpeed);
58          rb.linearVelocity = velocity;
59      }

```

```

60      1 reference
61      private void RotatePlayer()
62      {
63          Vector3 forward = cameraTransform != null ? cameraTransform.forward : Vector3.forward;
64          Vector3 right = cameraTransform != null ? cameraTransform.right : Vector3.right;
65
66          forward.y = 0f;
67          right.y = 0f;
68          forward.Normalize();
69          right.Normalize();
70
71          Vector3 move = forward * moveInput.y + right * moveInput.x;
72
73          if (move.sqrMagnitude > 0.001f)
74          {
75              Quaternion targetRotation = Quaternion.LookRotation(move);
76              Quaternion newRotation = Quaternion.Slerp(
77                  rb.rotation, targetRotation, rotationSpeed * Time.fixedDeltaTime);
78
79              rb.MoveRotation(newRotation);
80          }
81      }
82
83      private void UpdateAnimator()
84      {
85          if (animator == null) return;
86
87          Vector2 animInput = Vector2.zero;
88
89          if (moveInput.sqrMagnitude > 0.001f)
90          {
91              float locomotionAmount = isSprinting ? 1f : 0.5f;
92
93              animInput.x = Mathf.Abs(moveInput.x) > 0.01f ? Mathf.Sign(moveInput.x) * locomotionAmount : 0f;
94              animInput.y = Mathf.Abs(moveInput.y) > 0.01f ? Mathf.Sign(moveInput.y) * locomotionAmount : 0f;
95          }
96
97          currentAnimX = Mathf.SmoothDamp(
98              currentAnimX, animInput.x, ref animXVelocity, animationSmooth);
99
100         currentAnimY = Mathf.SmoothDamp(
101             currentAnimY, animInput.y, ref animYVelocity, animationSmooth);
102
103         animator.SetFloat("Horizontal", currentAnimX);
104         animator.SetFloat("Vertical", currentAnimY);
105     }

```

- ii. O segundo, que interage como novo sistema de input do Unity é o “PlayerInputHandler.cs”:

```

1 using UnityEngine;
2 using UnityEngine.InputSystem;
3
4 0 references
5 public class PlayerInputHandler : MonoBehaviour
6 {
7     6 references
8     private PlayerMovement movement;
9     3 references
10    private PlayerInteraction interaction;
11    3 references
12    public PlayerCombat combat;
13

```

```

14    private void Awake()
15    {
16        movement = GetComponent<PlayerMovement>();
17        interaction = GetComponent<PlayerInteraction>();
18        combat = GetComponent<PlayerCombat>();
19    }
20
21    0 references
22    public void OnMove(InputAction.CallbackContext context)
23    {
24        if (movement != null) movement.SetMoveInput(context.ReadValue<Vector2>());
25    }
26
27    0 references
28    public void OnSprint(InputAction.CallbackContext context)
29    {
30        if (movement != null)
31        {
32            if (context.performed) movement.SetSprint(true);
33            else if (context.canceled) movement.SetSprint(false);
34        }
35    }
36
37    0 references
38    public void OnInteract(InputAction.CallbackContext context)
39    {
40        if (context.performed && interaction != null) interaction.TryInteract();
41    }
42
43    0 references
44    public void OnAttack(InputAction.CallbackContext context)
45    {
46        if (combat == null) return;
47        if (context.performed) combat.Attack();
48    }
49 }

```

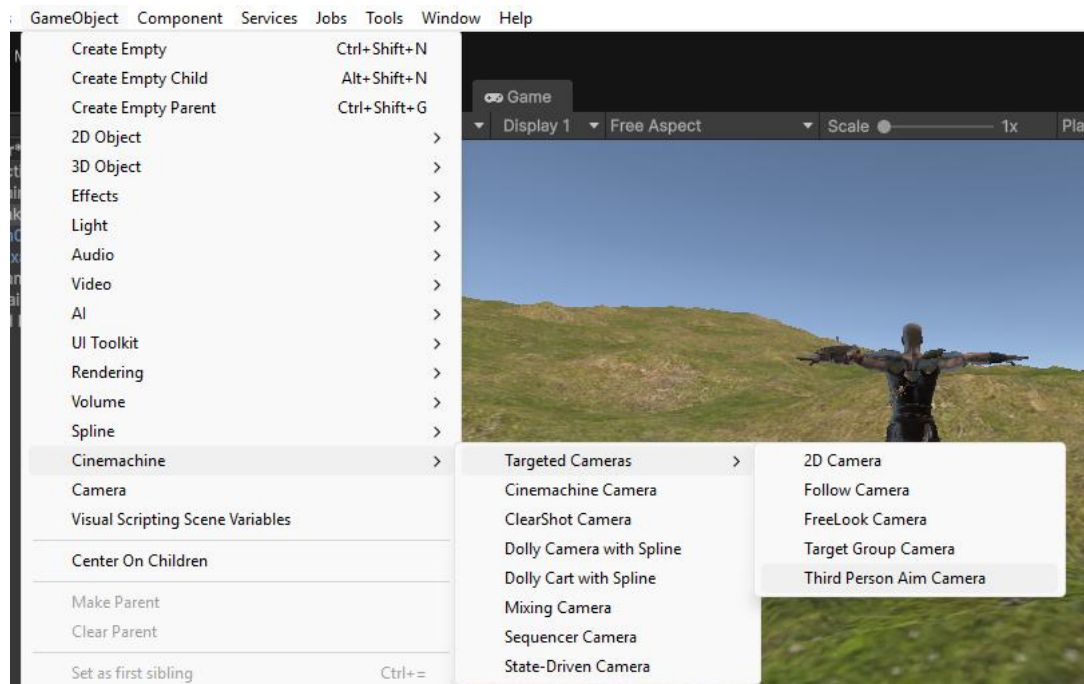
iii. O terceiro que é de combate, "PlayerCombat.cs":


```

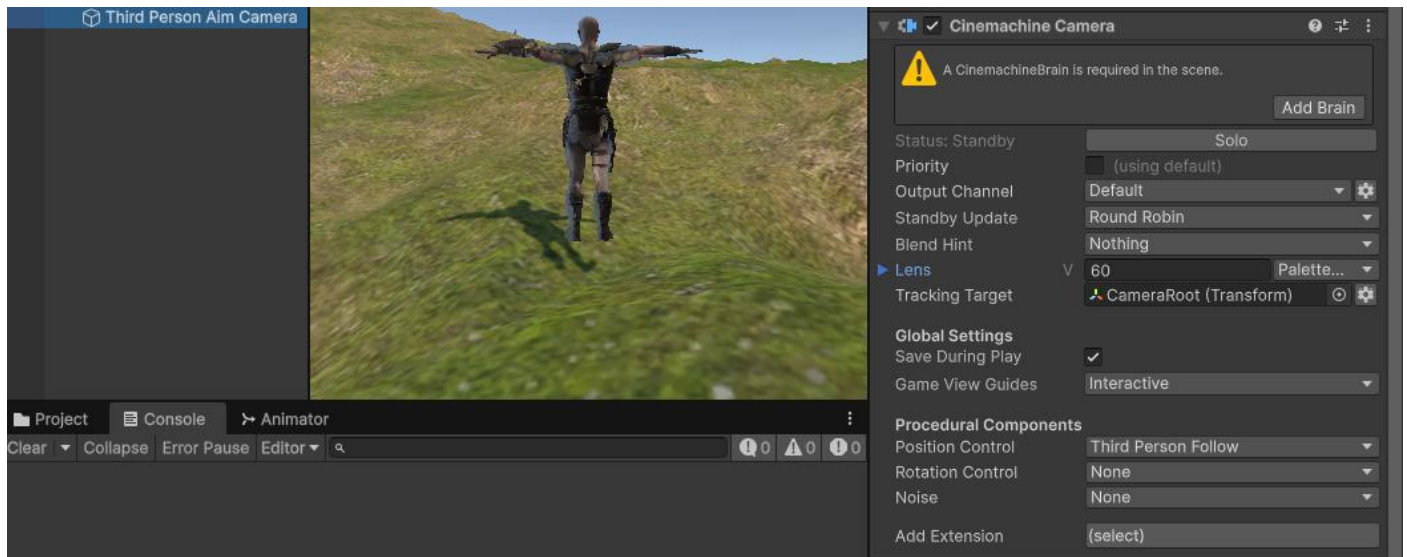
1  using UnityEngine;
2
3  0 references
4  public class PlayerCombat : MonoBehaviour
5  {
6      2 references
7      public Animator animator;
8      0 references
9      public void Attack()
10     {
11         if (animator == null)
12         {
13             Debug.LogError("Animator NULL no PlayerCombat");
14             return;
15         }
16         animator.SetTrigger("Attack");
17     }
18 }

```

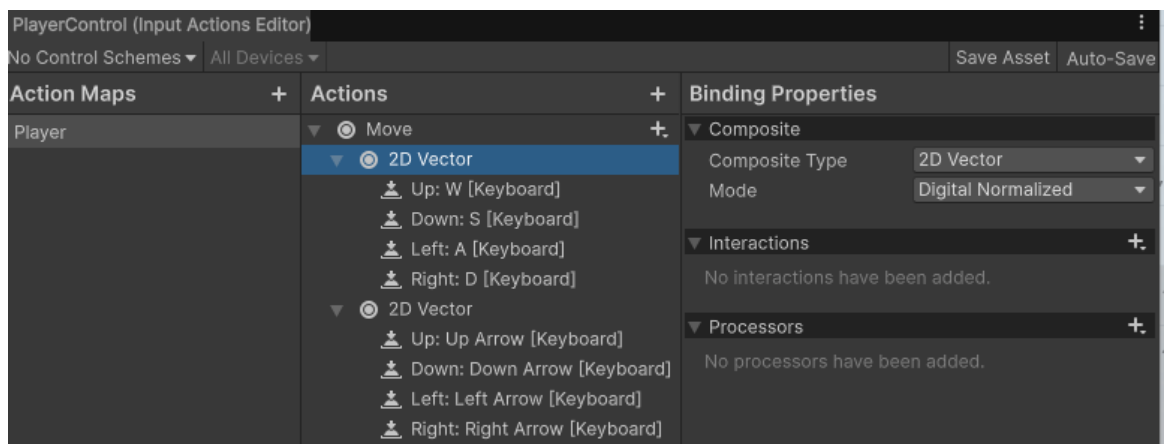
- r. Anexe os três scripts ao Player
 - s. Salve a Cena, antes de configurarmos o Cinemachine da Câmera.
3. Carregue o Cinemachine, desde o Package Manager.
- a. Na Hierarchy, crie uma Third Person Aim Camera:



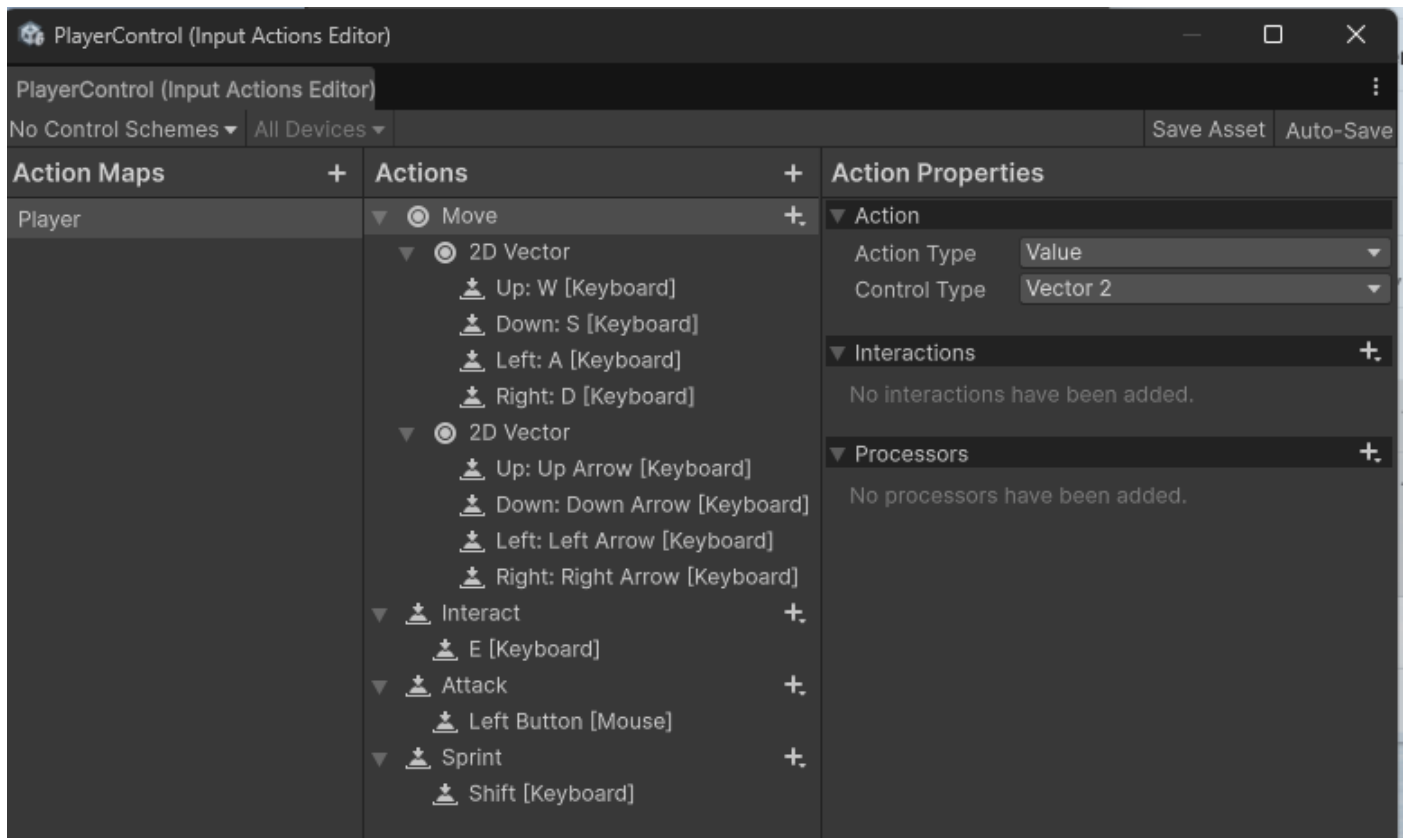
- b. Mova Main Camera para "fora" do Player, colocando-a como primeiro item da Hierarchy.
- c. Crie um GameObject Empty, filho do Player, e renomeie-o de CameraRoot.
- d. Selecionando este Third Person Aim Camera, associe Tracking Target a esta CameraRoot:



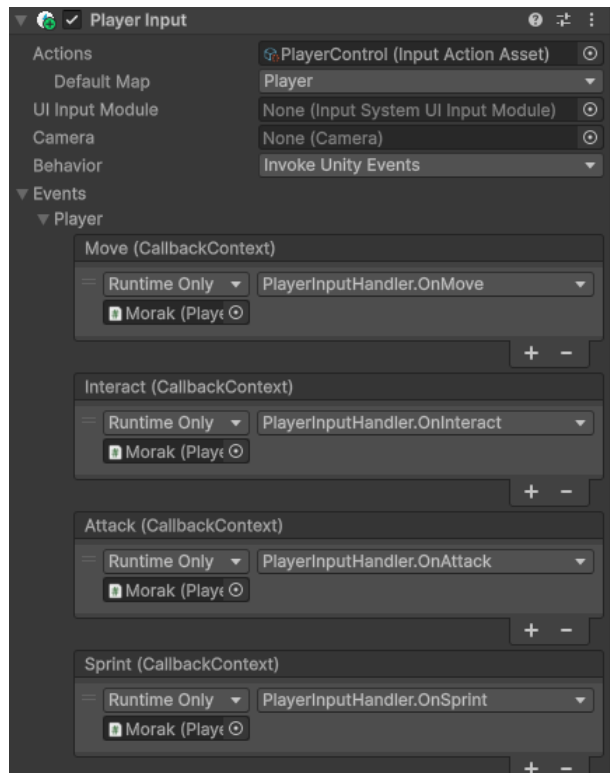
4. Vamos configurar o novo controlador de entrada, para substituir o padrão que utilizamos.
 - a. Na aba Project, > Create > Input Actions, renomeie de PlayerControl.
 - b. Em Action Maps, tecle + e chame de Player
 - c. Em Actions, tecle + e crie "Move"
 - i. Em Move tecle + Add Binding -> Composite
 1. Em Action Properties, Composite Type 2D Vector, Mode: Digital Normalized
 - a. Aparece 2D Vector em Move e para cada Up, Down, Left, Right, selecione W, S, A, D respectivamente
 2. + Add Binding -> Composite e repita para incluir as teclas direcionais



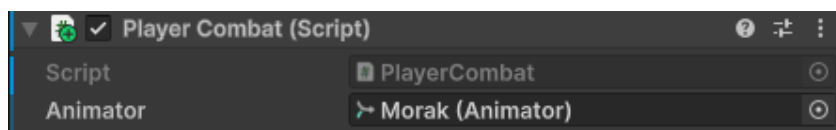
- ii. Em Actions +, crie Interact:
 1. + em Interact e Add Binding, e em Action Properties Action Type Button,
 - a. Selecione E (Keyboard)
 - iii. Em Actions +, crie Attack:
 1. + em Interact e Add Binding, e em Action Properties Action Type Button,
 - a. Selecione Left Button (Mouse)
 - iv. Em Actions +, crie Sprint:
 1. + em Interact e Add Binding, e em Action Properties Action Type Button,
 - a. Selecione Shift (Keyboard)
- d. A aparência deve ser algo assim:



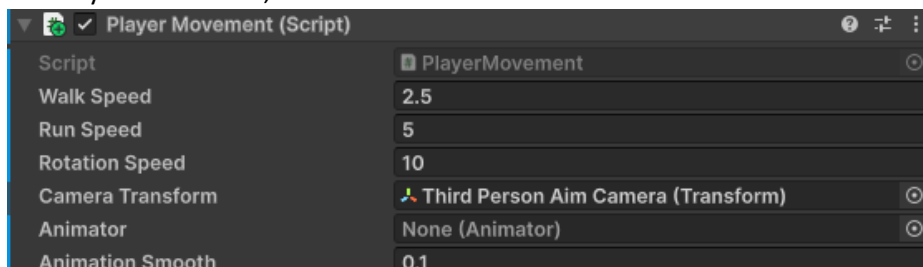
- i. No Player Control, “Save Asset”
- e. Na pasta Project, selecione o PlayerControl e no Inspector, selecione “Generate C Class”
- f. No Player, > Add Component > Player Input
 - i. Em Actions, associe este novo PlayerControl
 - ii. Em Behaviour: Invoke Unity Events
 - iii. Em Events, abra Player:
 - 1. Em todos os 4 eventos associe o Player (no nosso caso Morak) e depois o método correspondente: Move -> OnMove; Interact -> OnInteract, Attack -> OnAttack e Sprint -> OnSprint:



- g. Em Player Combat, associe o Animator do Player:



- h. Em Player Movement, associe a Camera Transform. Third Person Aim Camera:



- i. No Animator Controller, crie um parâmetro Trigger "Attack".
- j. Salve o projeto e a cena. Rode e teste, verifique:
- se o Player se movimenta com as teclas WASD e as direcionais
 - se a câmera está bem posicionada como 3ª Pessoa.

1. Ajuste-a no Third Person Aim Camera se necessário.

5. Adicionando uma animação de Ataque:

- No Animator Controller, adicione na Base Layer a animação "HumanM@Attack2H01" (pasta 2H das animações importadas).
- Do estado "Movimentação" crie uma transição, conditions "Attack" e retire a seleção de Has Exit Time
- Deste estado "HumanM@Attack2H01" -> "Movimentação" deixe Has Exit Time selecionado.
- Salve a cena. Rode e teste, verificando se com o botão do mouse esquerdo ocorre a animação de ataque.

6. Configurando segunda cena do Mundo do Game

- a) Vamos criar uma nova Scene “Vilarejo”, > File > New Scene > Vilarejo, salva na pasta \Assets\Scenes, de acordo com o rascunho:
- b)

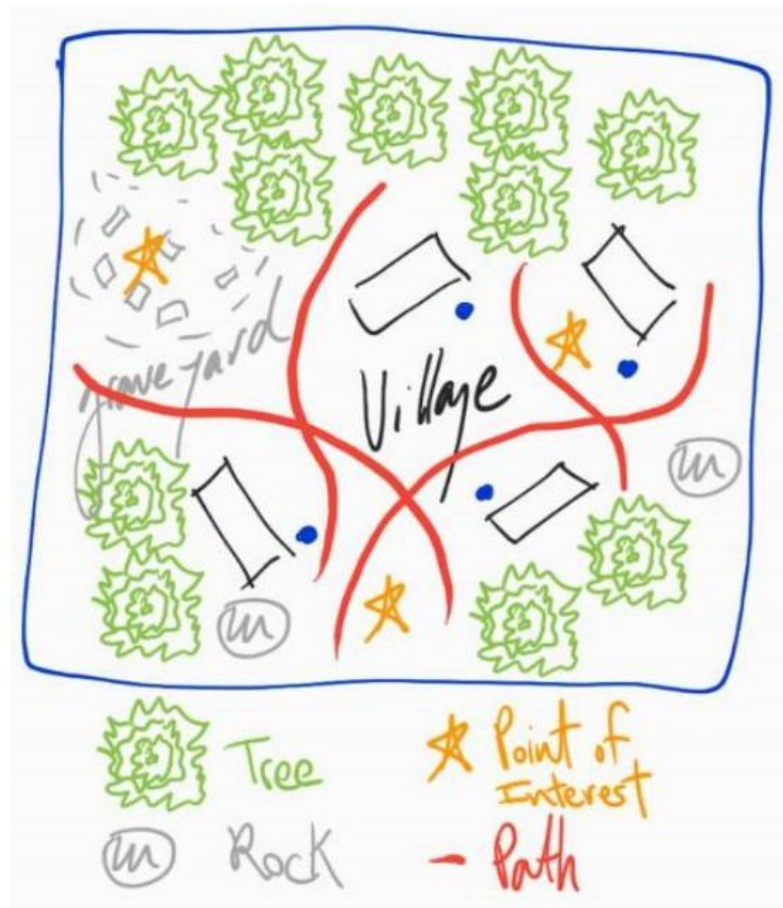


Figura 1. Rascunho da cena Vilarejo

- c) Repita os procedimentos do Lab5, mas agora com o rascunho anterior, tendo em conta o Cemitério e o Vilarejo. No meu projeto o Terrain tinha 1000 x 1000, e assim coloquei-o em XYZ = (-500, 0, -500) para centralizar. Usei também o modelo Pampa, com quase nada de Height, para não ter aclives e declives.
- d) Coloque um Skybox diferente do da primeira Scene.
- e) Fica mais ou menos como nas figuras 2 e 3, a entrada do vilarejo e depois o cemitério:

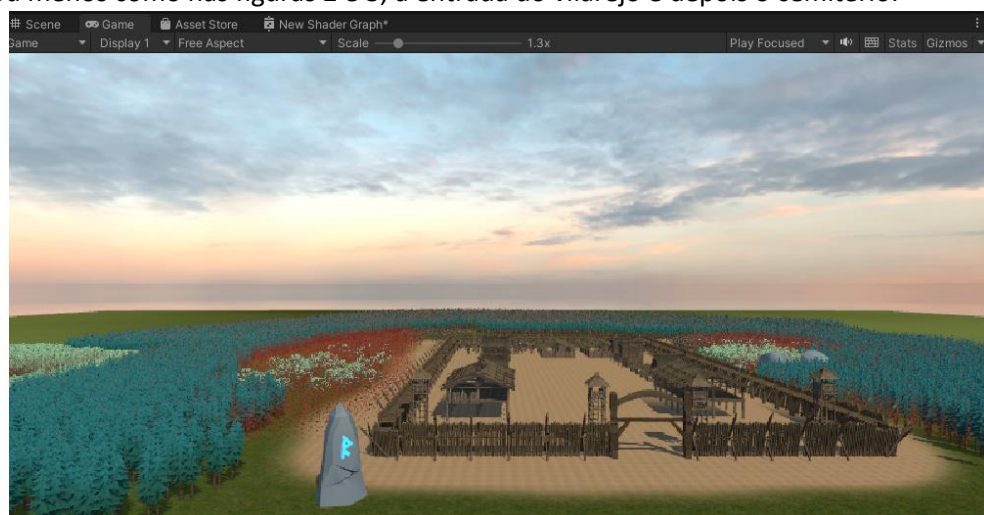


Figura 2. Portão de entrada do Vilarejo



Figura 3. Detalhe do cemitério

- f) A ideia é de que nosso Personagem sai da Scene “Despertar” e inicie esta Scene “Vilarejo”, passando primeiro em frente ao Cemitério, conforme a figura 4.

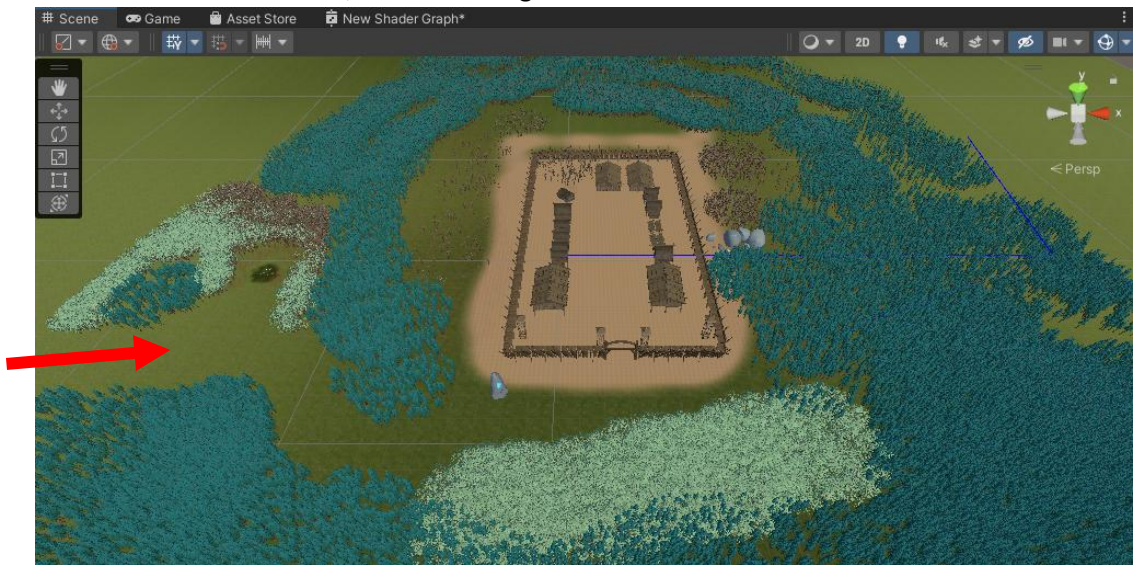


Figura 4. Entrada do Player na Cena

-X-X-X-